Lab Report: Week 4 Session B
Integration & Flow Practice

Mahmudul Hasan (Student ID: 4125999049)

May 21, 2026

Abstract

This report documents Week 4 Session B on **integration and flow**: wiring a rainfall monitoring pipeline (weather client, SCS-CN processor, alerts, CSV storage) with OpenCode Desktop, then hardening errors and building a Streamlit dashboard. Work was done in `week4_session_b/` using Open-Meteo by default (no API key). Exercise 1 integrated modules with logging, retries, and boundary exceptions; Exercise 2 added `ConfigError`, validated configuration, and tested CONFIG ERROR / DEGRADED paths; Exercise 3 delivered a dashboard reading `data/monitoring_log.csv`. Evidence: seven screenshots (OpenCode, terminal, Streamlit) and source in Appendix ??–??.

1 Introduction

Integrated systems connect independent components through clear interfaces and error boundaries. This session practiced **staying in flow with AI**: one feature thread at a time (integrate, then errors, then UI), paste interfaces first, and verify with small terminal loops after each step. Week 4 Session A refactored a single legacy script; Session B builds a multi-module monitor aligned with earlier SCS-CN work (Week 3) but focused on orchestration and resilience.

2 Objectives

- Integrate weather fetch, rainfall processing, alerts, and CSV storage.
- Implement robust error handling (config, API, storage).
- Build a Streamlit monitoring dashboard.
- Document data flow and AI-assisted workflow in `prompt_log.md`.

3 Methods

3.1 Architecture

Four components plus configuration:

1. `weather_client.py` — precipitation via Open-Meteo or OpenWeather (`OPENWEATHER_API_KEY`)
2. `rainfall_processor.py` — SCS-CN runoff (`process_reading`)
3. `alert_system.py` — OK / WATCH / ALERT thresholds
4. `storage.py` — append rows to `data/monitoring_log.csv`
5. `config.py` — `load_settings()` from environment

Orchestration: `main.py` `run_once()`. Visualization: `dashboard.py` via `python3 -m streamlit run`.

3.2 Exercise 1: Integration

Prompt `prompt_integration.txt` enhanced `main.py` and `weather_client.py`: pipeline logging, `WeatherAPIError` → DEGRADED (exit 1), `StorageError` → WARNING (non-fatal), exponential backoff retries (`api_max_retries`).

3.3 Exercise 2: Error handling

Prompt `prompt_errors.txt` added `ConfigError`, moved env parsing into `load_settings()`, JSON parse guards, and `OSError` handling in storage. Tested `MONITOR_LAT=abc`, `MONITOR_LAT=999`, and `API_TIMEOUT_SEC=0.001`.

3.4 Exercise 3: Dashboard

Prompt `prompt_dashboard.txt` produced a wide-layout Streamlit UI: metrics, color-coded alerts, line chart, recent table, empty-CSV handling.

3.5 Exercise 4: Verification

Checklist: cold-start `main.py`, failure paths, CSV growth, dashboard reflects latest row.

4 Results

4.1 File inventory

Table 1: Submitted files in `week4_session_b/`.

File	Role
`main.py`	Integrated pipeline
`weather_client.py`	API + retry
`rainfall_processor.py`	SCS-CN
`alert_system.py`	Threshold alerts
`storage.py`	CSV append
`config.py`	Env settings + `ConfigError`
`dashboard.py`	Streamlit UI
`data/monitoring_log.csv`	History
`prompt_*.txt`, `prompt_log.md`	AI workflow

4.2 Exercise 1: Integration

Figure **??** shows OpenCode integration; Figure **??** shows a successful run.

4.3 Exercise 2: Errors and resilience

Three OpenCode screenshots (Figures **??**–**??**) document one long Exercise 2 session. Figure **??** shows terminal tests (e.g. retries and DEGRADED).

Table **??** summarizes failure handling.

4.4 Exercise 3: Streamlit dashboard

Figure **??** shows the monitoring UI after `python3 -m streamlit run dashboard.py` (Streamlit installed with `pip install --user`; module invocation used because `streamlit` was not on PATH).

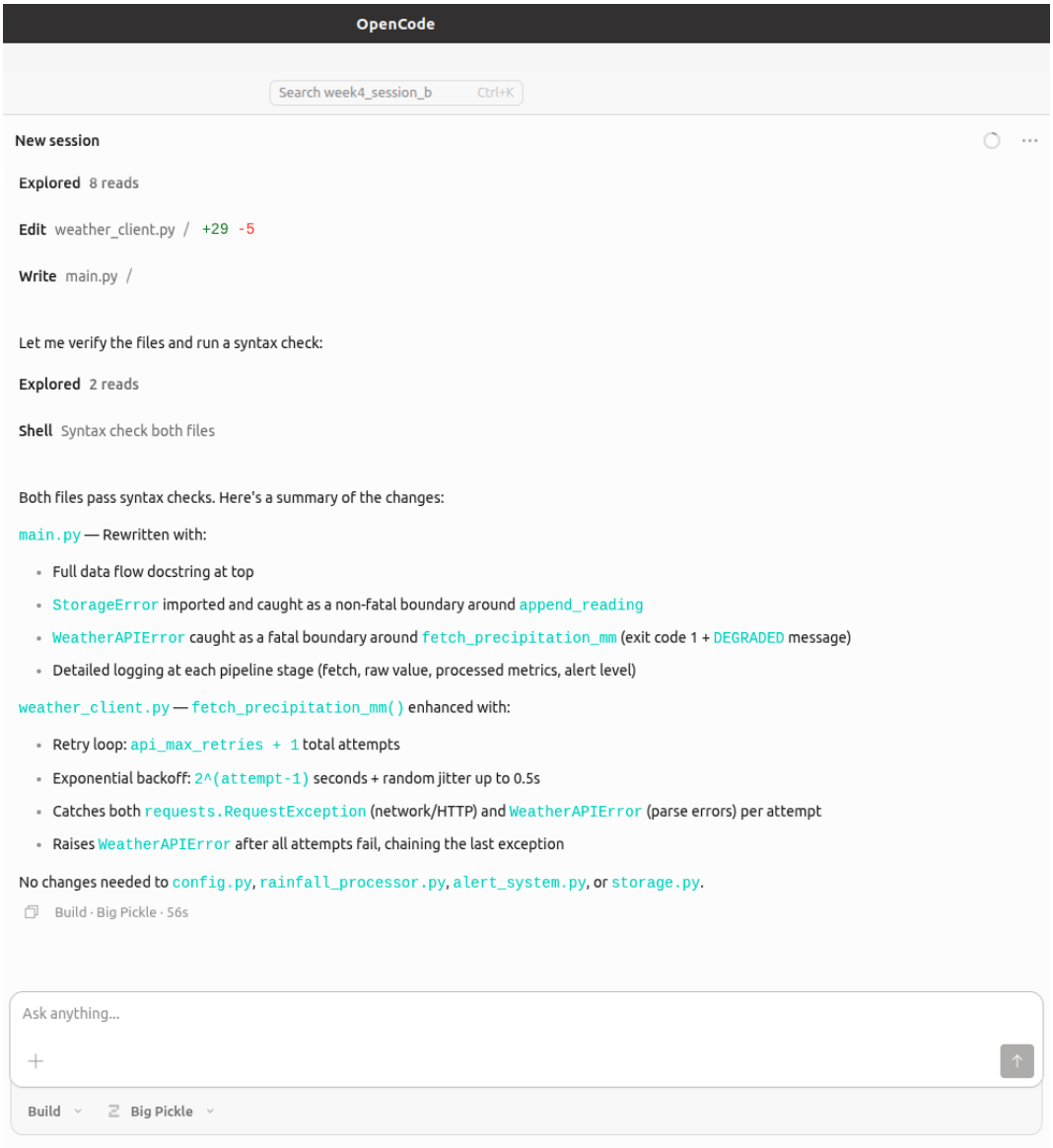

Figure 1: OpenCode Exercise 1: integration prompt and pipeline changes.

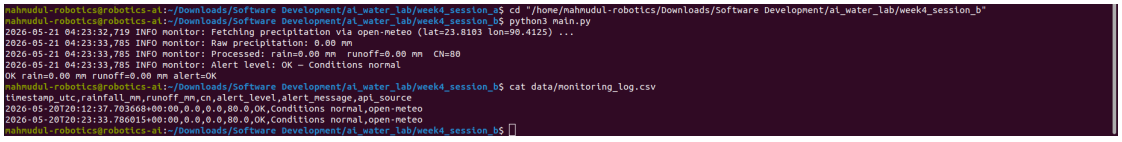

Figure 2: Terminal Exercise 1: logged fetch/process/alert; `OK rain=... alert=OK`; CSV append.

Table 2: Error boundaries and user-visible messages.

Boundary	Exception	User message / exit
Config	`ConfigError`	`CONFIG ERROR`, exit 1
Weather	`WeatherAPIError`	`DEGRADED`, exit 1
Storage	`StorageError`	`WARNING`, continue

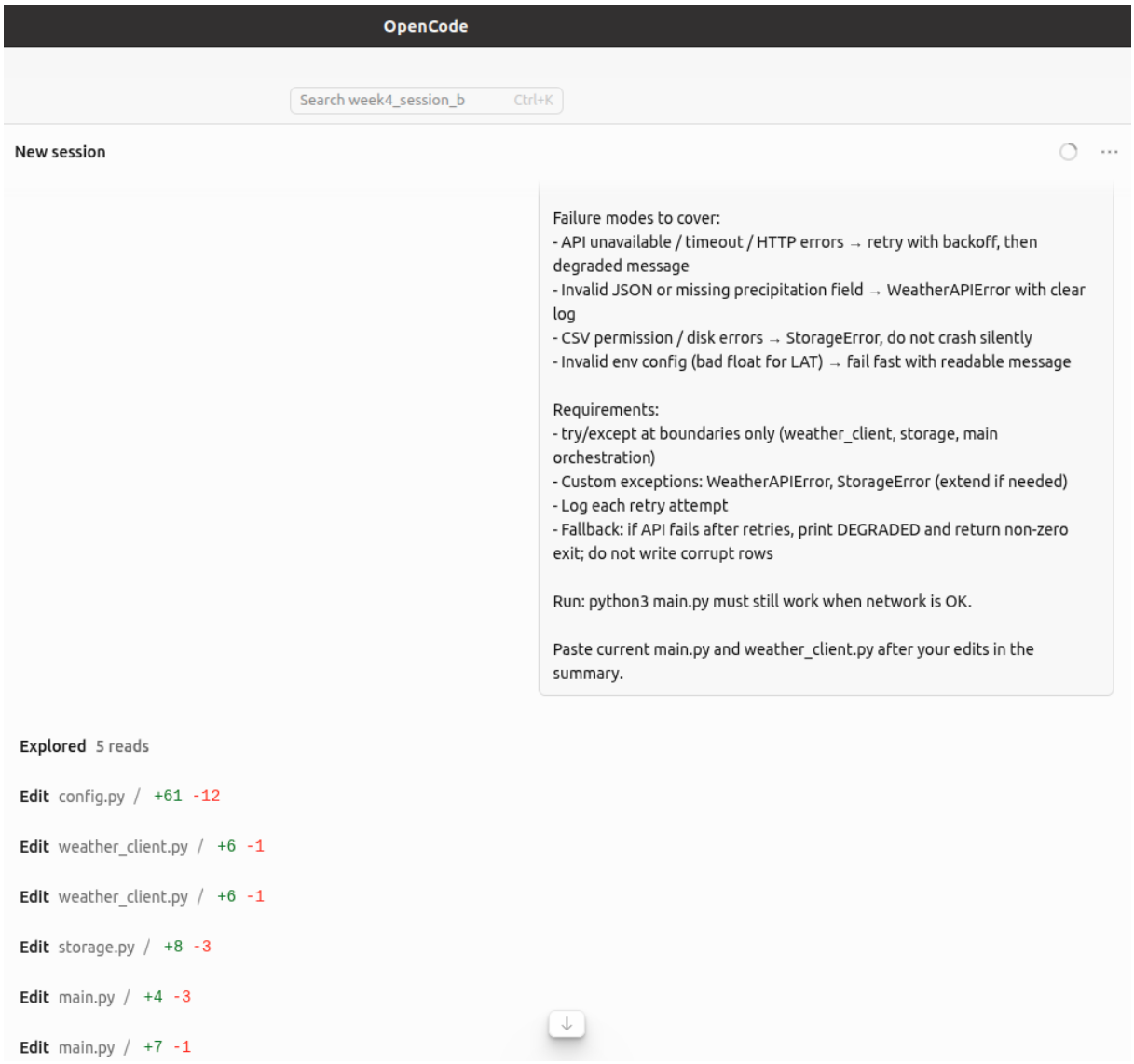

Figure 3: OpenCode Exercise 2 (1/3): error-handling prompt and `config.py` changes.

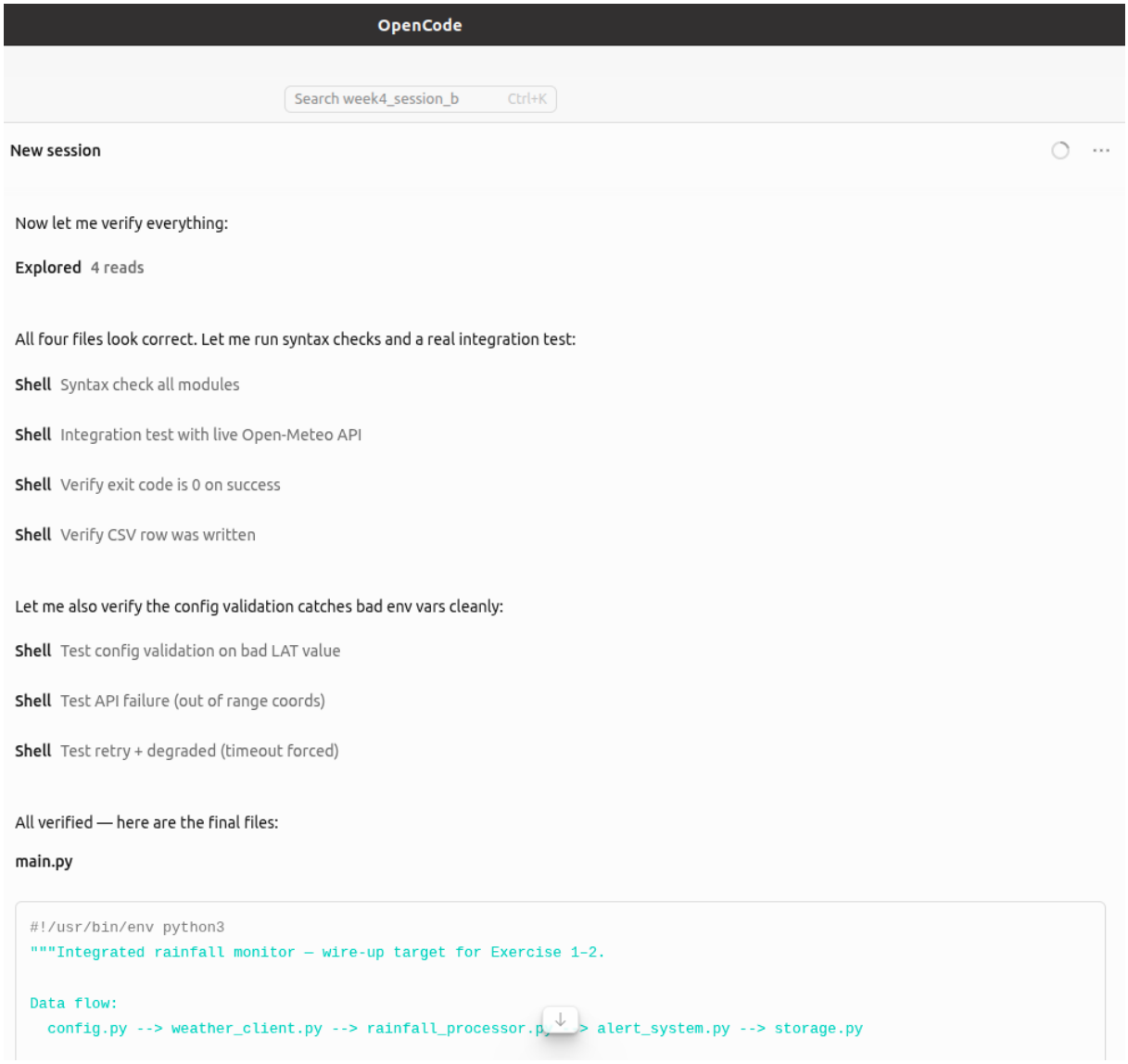

Figure 4: OpenCode Exercise 2 (2/3): `weather_client` / `storage` guards.

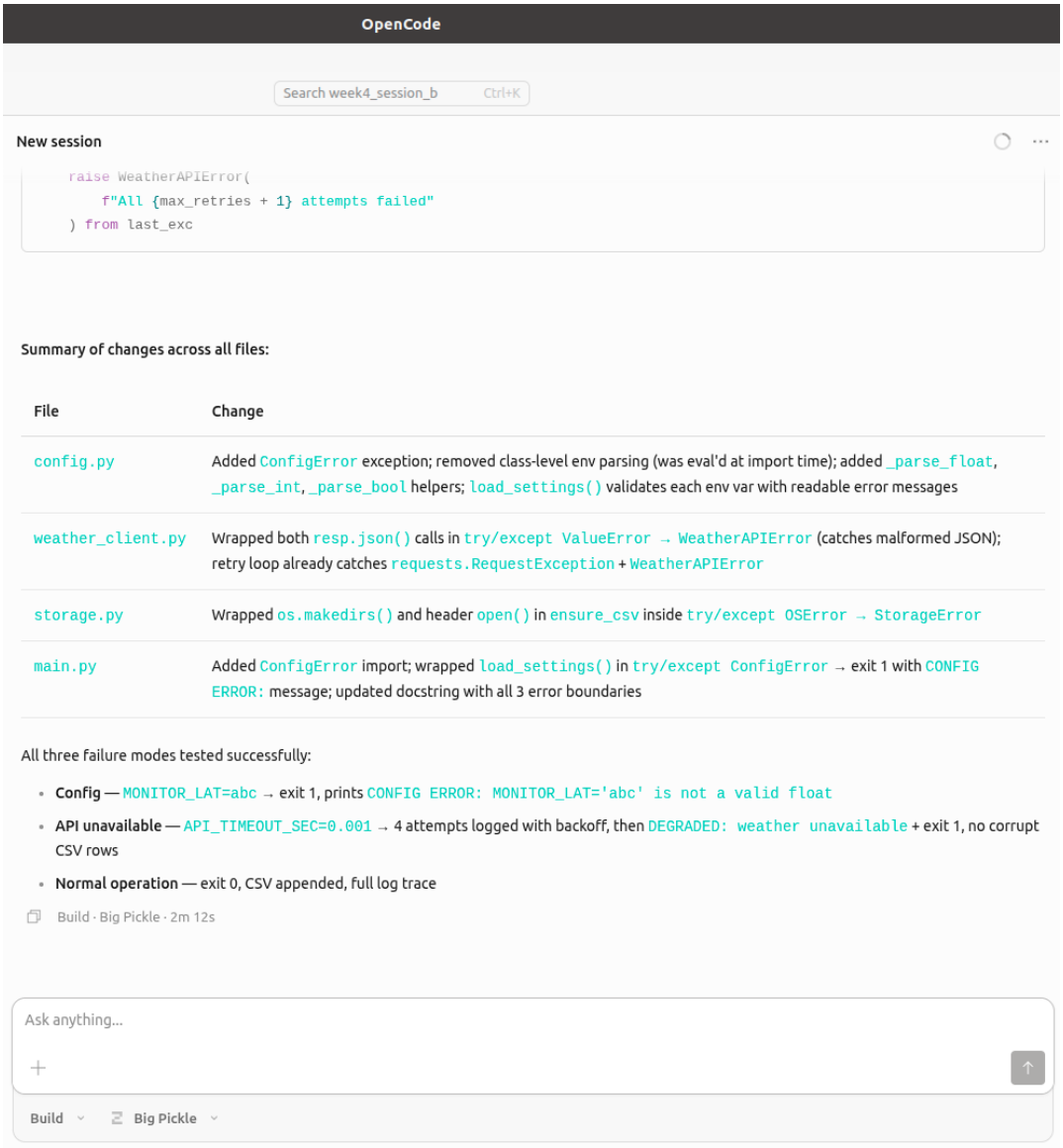

Figure 5: OpenCode Exercise 2 (3/3): verification summary (config, API, normal paths).

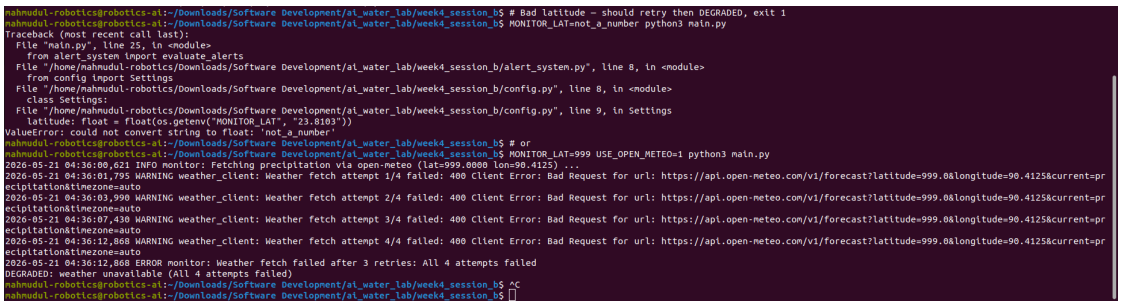

Figure 6: Terminal Exercise 2: `CONFIG ERROR` or `DEGRADED` after retries (failure-path tests).

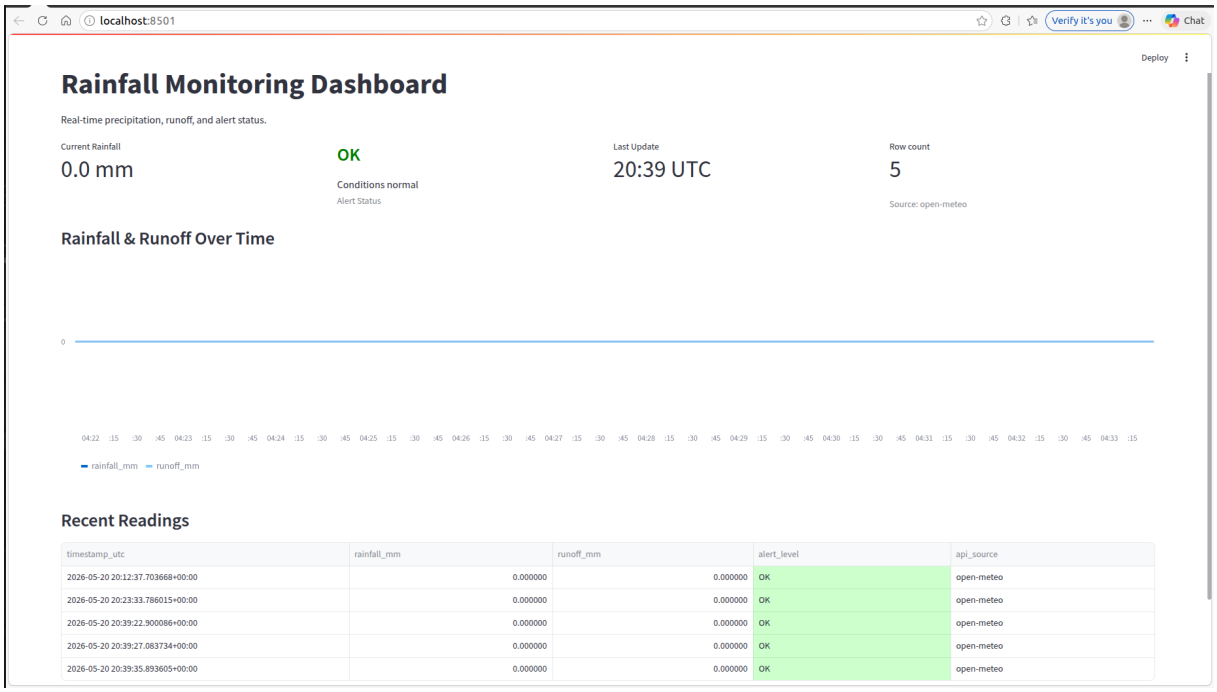

Figure 7: Streamlit dashboard: current rainfall, alert status, chart, health metrics.

5 Analysis and validation

Data flow: A single `run_once()` call exercises all components; CSV is the handoff to the dashboard—no duplicate API calls in the UI.

Open-Meteo vs. lab template: The handout mentions OpenWeather; implementation defaults to Open-Meteo (keyless prototyping) with optional OpenWeather via environment—consistent with Week 2 project choices.

Resilience: Failed API runs do not append corrupt rows when the pipeline exits before `append_reading`. Config errors fail before network I/O.

Flow discipline: Separate prompts (`integration`, `errors`, `dashboard`) matched the lab's one-thread-per-feature guidance and reduced context loss.

6 Discussion

1. Exercise 1 already included retries; Exercise 2 added config validation that fixed import-time `ValueError` on bad `MONITOR_LAT`.
2. Multiple OpenCode screenshots for Exercise 2 reflect scroll length, not separate prompts.
3. `python3 -m streamlit` is required when the Streamlit CLI is not on PATH.
4. Dashboard reads CSV only—appropriate separation for a monitoring view.

7 Problems encountered

`streamlit: command not found` resolved with `python3 -m streamlit run`. First-time Streamlit email prompt required pressing Enter on a blank line. Pip reported a jupyterlab version conflict unrelated to this lab.

8 Lessons learned

Integrate with explicit boundary messages (`CONFIG ERROR` vs. `DEGRADED`). Test three failure modes after AI edits. Use module-form CLI invocations on Linux lab machines. Screenshot terminal and UI as evidence alongside `prompt_log.md`.

9 Conclusion

Week 4 Session B objectives were met: integrated rainfall monitor, hardened error paths, Streamlit dashboard, and documented AI workflow. Submitted: `week4_session_b/` sources, `prompt_log.md`, and seven screenshots in `Lab-reports/`.

A Orchestration: `main.py`

main.py (full file)

```
#!/usr/bin/env python3
"""Integrated rainfall monitor | wire-up target for Exercise 1{2.

Data flow:
  config.py --> weather_client.py --> rainfall_processor.py --> alert_system.py -->
  ↪  storage.py

Flow details:
  1. load_settings()        | reads env vars via config.Settings
  2. fetch_precipitation_mm() | Open-Meteo (default) or OpenWeather API,
                              with exponential-backoff retry (settings.api_max_retries)
  3. process_reading()      | SCS-CN runoff calculation
  4. evaluate_alerts()      | thresholds -> OK / WATCH / ALERT
  5. append_reading()       | writes CSV row at settings.csv_path

Error boundaries:
  - Config error (bad env var) -> log ERROR, print CONFIG ERROR, exit 1
  - Weather API failure         -> log ERROR, print DEGRADED, exit 1
  - Storage write failure       -> log ERROR, print WARNING, continue (non-fatal)
"""

from __future__ import annotations

import logging
import sys

from alert_system import evaluate_alerts
from config import ConfigError, load_settings
from rainfall_processor import process_reading
from storage import StorageError, append_reading
from weather_client import WeatherAPIError, fetch_precipitation_mm

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s",
)
logger = logging.getLogger("monitor")


def run_once() -> int:
    try:
        settings = load_settings()
    except ConfigError as exc:
```

```
        logger.error("Configuration error: %s", exc)
        print(f"CONFIG ERROR: {exc}")
        return 1

    api_source = "open-meteo" if settings.use_open_meteo else "openweather"

    logger.info("Fetching precipitation via %s (lat=%.4f lon=%.4f) ...",
                api_source, settings.latitude, settings.longitude)

    try:
        rainfall_mm = fetch_precipitation_mm(settings)
    except WeatherAPIError as exc:
        logger.error("Weather fetch failed after %d retries: %s",
                     settings.api_max_retries, exc)
        print(f"DEGRADED: weather unavailable ({exc})")
        return 1

    logger.info("Raw precipitation: %.2f mm", rainfall_mm)

    processed = process_reading(rainfall_mm, cn=settings.default_cn)
    logger.info("Processed: rain=%.2f mm  runoff=%.2f mm  CN=%.0f",
                processed["rainfall_mm"], processed["runoff_mm"],
                processed["cn"])

    alert = evaluate_alerts(processed, settings)
    logger.info("Alert level: %s | %s", alert.level, alert.message)

    try:
        append_reading(
            settings, processed, alert.level, alert.message, api_source,
        )
    except StorageError as exc:
        logger.error("Storage write failed: %s", exc)
        print(f"WARNING: could not persist reading ({exc})")

    print(
        f"OK rain={processed['rainfall_mm']:.2f} mm "
        f"runoff={processed['runoff_mm']:.2f} mm alert={alert.level}"
    )
    return 0


if __name__ == "__main__":
    sys.exit(run_once())
```

B Configuration: `config.py`

config.py (full file)

```
"""Configuration for rainfall monitoring integration (Week 4 Session B).

Validates all environment variables at load time for fast failure on bad config.
"""

import os
from dataclasses import dataclass


class ConfigError(Exception):
    """Invalid environment configuration."""
```

```
@dataclass
class Settings:
    latitude: float
    longitude: float
    openweather_api_key: str
    use_open_meteo: bool
    rainfall_alert_mm: float
    runoff_alert_mm: float
    default_cn: int
    csv_path: str
    api_timeout_sec: float
    api_max_retries: int


def _parse_float(key: str, default: float) -> float:
    raw = os.getenv(key)
    if raw is None:
        return default
    try:
        return float(raw)
    except ValueError:
        raise ConfigError(
            f"{key}={raw!r} is not a valid float"
        )


def _parse_int(key: str, default: int) -> int:
    raw = os.getenv(key)
    if raw is None:
        return default
    try:
        return int(raw)
    except ValueError:
        raise ConfigError(
            f"{key}={raw!r} is not a valid integer"
        )


def _parse_bool(key: str, default: bool) -> bool:
    raw = os.getenv(key)
    if raw is None:
        return default
    return raw == "1"


def load_settings() -> Settings:
    return Settings(
        latitude=_parse_float("MONITOR_LAT", 23.8103),
        longitude=_parse_float("MONITOR_LON", 90.4125),
        openweather_api_key=os.getenv("OPENWEATHER_API_KEY", ""),
        use_open_meteo=_parse_bool("USE_OPEN_METEO", True),
        rainfall_alert_mm=_parse_float("RAINFALL_ALERT_MM", 20.0),
        runoff_alert_mm=_parse_float("RUNOFF_ALERT_MM", 15.0),
        default_cn=_parse_int("DEFAULT_CN", 80),
        csv_path=os.getenv("MONITOR_CSV", "data/monitoring_log.csv"),
        api_timeout_sec=_parse_float("API_TIMEOUT_SEC", 10),
        api_max_retries=_parse_int("API_MAX_RETRIES", 3),
    )
```

C Weather client: `weather_client.py`

weather_client.py (full file)

```
"""Weather API client | Open-Meteo (no key) or OpenWeather (env key).

Retry with exponential backoff is applied in the unified entry point.
"""

from __future__ import annotations

import logging
import random
import time
from typing import Any, Dict, Optional

import requests

from config import Settings

logger = logging.getLogger(__name__)


class WeatherAPIError(Exception):
    """Raised when weather data cannot be fetched or parsed."""


def fetch_open_meteo_precipitation_mm(settings: Settings) -> float:
    """Current-hour precipitation (mm) from Open-Meteo."""
    url = "https://api.open-meteo.com/v1/forecast"
    params = {
        "latitude": settings.latitude,
        "longitude": settings.longitude,
        "current": "precipitation",
        "timezone": "auto",
    }
    resp = requests.get(url, params=params, timeout=settings.api_timeout_sec)
    resp.raise_for_status()
    try:
        data = resp.json()
    except ValueError as exc:
        raise WeatherAPIError(
            f"Open-Meteo invalid JSON: {exc}"
        ) from exc
    current = data.get("current") or {}
    precip = current.get("precipitation")
    if precip is None:
        raise WeatherAPIError("Open-Meteo response missing current.precipitation")
    return float(precip)


def fetch_openweather_precipitation_mm(settings: Settings) -> float:
    """Rain in last 1h (mm) from OpenWeather | requires OPENWEATHER_API_KEY."""
    if not settings.openweather_api_key:
        raise WeatherAPIError("OPENWEATHER_API_KEY not set")
    url = "https://api.openweathermap.org/data/2.5/weather"
    params = {
        "lat": settings.latitude,
        "lon": settings.longitude,
        "appid": settings.openweather_api_key,
        "units": "metric",
    }
    resp = requests.get(url, params=params, timeout=settings.api_timeout_sec)
```

```
    resp.raise_for_status()
    try:
        data = resp.json()
    except ValueError as exc:
        raise WeatherAPIError(
            f"OpenWeather invalid JSON: {exc}"
        ) from exc
    rain = (data.get("rain") or {}).get("1h")
    if rain is None:
        return 0.0
    return float(rain)


def fetch_precipitation_mm(settings: Settings) -> float:
    """Unified entry: Open-Meteo by default, with retry + exponential backoff."""
    max_retries = settings.api_max_retries
    last_exc = None

    for attempt in range(1, max_retries + 2):
        try:
            if settings.use_open_meteo:
                return fetch_open_meteo_precipitation_mm(settings)
            return fetch_openweather_precipitation_mm(settings)
        except (requests.RequestException, WeatherAPIError) as exc:
            last_exc = exc
            logger.warning(
                "Weather fetch attempt %d/%d failed: %s",
                attempt, max_retries + 1, exc,
            )
            if attempt <= max_retries:
                delay = 2.0 ** (attempt - 1)
                jitter = random.uniform(0, 0.5)
                time.sleep(delay + jitter)

    raise WeatherAPIError(
        f"All {max_retries + 1} attempts failed"
    ) from last_exc
```

D Dashboard: `dashboard.py`

dashboard.py (full file)

```
"""Streamlit dashboard for rainfall monitoring | reads data/monitoring_log.csv."""

from __future__ import annotations

import pandas as pd
import streamlit as st

from config import load_settings

st.set_page_config(
    page_title="Rainfall Monitor",
    page_icon="",
    layout="wide",
)

st.title("Rainfall Monitoring Dashboard")
st.markdown("Real-time precipitation, runoff, and alert status.")

settings = load_settings()
```

```
csv_path = settings.csv_path

try:
    df = pd.read_csv(csv_path, parse_dates=["timestamp_utc"])
except FileNotFoundError:
    st.error(f"CSV not found at {csv_path}. Run `python3 main.py` first.")
    st.stop()

if df.empty:
    st.warning("CSV is empty | no readings recorded yet.")
    col1, col2, _ = st.columns([1, 1, 2])
    with col1:
        st.metric("Row count", 0)
    with col2:
        st.metric("Last API source", "|")
    st.stop()

df = df.sort_values("timestamp_utc").reset_index(drop=True)
latest = df.iloc[-1]

alert_color_map = {"OK": "green", "WATCH": "orange", "ALERT": "red"}
alert_color = alert_color_map.get(latest["alert_level"], "grey")

col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric("Current Rainfall", f'{latest["rainfall_mm"]:.1f} mm')

with col2:
    st.markdown(
        f"<h3 style='color:{alert_color}; margin:0;'>"
        f"{latest['alert_level']}</h3>"
        f"<p style='margin:0;'>{latest['alert_message']}</p>",
        unsafe_allow_html=True,
    )
    st.caption("Alert Status")

with col3:
    st.metric("Last Update", latest["timestamp_utc"].strftime("%H:%M UTC"))

with col4:
    st.metric("Row count", len(df))
    st.caption(f"Source: {latest['api_source']}")

st.subheader("Rainfall & Runoff Over Time")
st.line_chart(
    df.set_index("timestamp_utc")[["rainfall_mm", "runoff_mm"]],
)

st.subheader("Recent Readings")
st.dataframe(
    df[["timestamp_utc", "rainfall_mm", "runoff_mm", "alert_level", "api_source"]]
    .tail(10)
    .style.applymap(
        lambda v: (
            "background-color: #ffcccc" if v == "ALERT" else
            "background-color: #ffe0b3" if v == "WATCH" else
            "background-color: #ccffcc" if v == "OK" else ""
        ),
        subset=["alert_level"],
    ),
    use_container_width=True,
    hide_index=True,
```

```
)
```

E Other modules

`rainfall_processor.py`, `alert_system.py`, and `storage.py` are included in the course submission folder `week4_session_b/` (see Table ??). Full copies are available in `week4_session_b_files/` on the report build machine.

F Submitted prompt log: `prompt_log.md`

prompt_log.md (full file)

```
# Prompt Log - Week 4 Session B

**Student:** Mahmudul Hasan (4125999049)
**Lab:** Integration & Flow Practice

---

## Integration

**Data flow (one sentence):** `load_settings`  `fetch_precipitation_mm` (Open-Meteo)
↪  `process_reading` (SCS-CN runoff)  `evaluate_alerts`  `append_reading`
↪  `data/monitoring_log.csv`.

**Hardest handoff between components:** Weather API  processor: retries must complete
↪  before alert/storage; storage errors must not mask weather failures.

**Prompt used (Exercise 1):** `prompt_integration.txt`

**OpenCode changes (Exercise 1):** `main.py` logging + error boundaries;
↪  `weather_client.py` retry with exponential backoff + jitter.

**Terminal (success):** `OK rain=0.00 mm runoff=0.00 mm alert=OK`  CSV row appended with
↪  `open-meteo`.

---

## Errors & resilience

**Prompt used (Exercise 2):** `prompt_errors.txt`

**OpenCode changes (Exercise 2):**
- `config.py`: `ConfigError`, `load_settings()` validates env (no parse at import time)
- `weather_client.py`: invalid JSON → `WeatherAPIError`
- `storage.py`: `OSError` → `StorageError` in `ensure_csv`
- `main.py`: `ConfigError`  `CONFIG ERROR` exit 1; weather  `DEGRADED` exit 1; storage
↪  `WARNING` non-fatal

**Failure modes tested:**

| Test | Command | Expected | Result |
|---|---|---|---|
| Bad config | `MONITOR_LAT=abc python3 main.py` | CONFIG ERROR, exit 1 | Yes  no
↪  traceback |
| Bad API coords | `MONITOR_LAT=999 python3 main.py` | 4 retries, DEGRADED, exit 1 | Yes
↪   no new bad row |
| API timeout | `API_TIMEOUT_SEC=0.001 python3 main.py` | retries, DEGRADED, exit 1 | Yes
↪  |
```

```
| Normal | `python3 main.py` | exit 0, CSV append | Yes |

**Note:** `MONITOR_LAT=not_a_number` crashed at import *before* Exercise 2 fix
↪  (class-level `float()`). After fix, invalid env is caught in `load_settings()` with a
↪  clear message.

**Still fragile (if any):** No cached last-good reading when API fails; dashboard pending
↪  (Exercise 3).

---

## Dashboard

**Prompt used (Exercise 3):** `prompt_dashboard.txt`

**OpenCode changes:** `dashboard.py`  metrics (rain, alert, timestamp, health),
↪  color-coded alert, line chart, last-10 table, empty/missing CSV handling.

**Run command (use module form if `streamlit` not on PATH):**

```bash
python3 -m pip install --user streamlit   # once
python3 -m streamlit run dashboard.py
```

Browser opens at http://localhost:8501 | screenshot for submission.

Flow

What kept you in flow / broke flow: Same OpenCode thread for Exercises 12; testing
↪ three env variants (`abc`, `999`, timeout) immediately after each change. Clear exit
↪ messages (`CONFIG ERROR` vs `DEGRADED`) made screenshots easy to label.

Verification (Exercise 4)

- [x] Cold start: `python3 main.py` works
- [x] API failure: DEGRADED, exit 1, logs show retries
- [x] Config failure: CONFIG ERROR, exit 1
- [x] CSV append on success path
- [x] Dashboard: `python3 -m streamlit run dashboard.py` (after install)

Lessons learned

Boundary errors at config, network, and storage kept `main.py` readable. Retries belong
↪ in the client module. Always re-test failure paths after AI changes import-time
↪ config parsing was a real bug until Exercise 2.
```